ASSIGNMENT NO : 11

NAME : SWARALI RAJKUMAR GHOLAP

ROLL NO : EN24107049

BATCH : TY-C

Title : Object Recognition using CNN (CIFAR-10 Dataset)

a. Train a CNN model to classify CIFAR-10 images into 10 object categories (airplane, cat,car, dog, etc.).

b. Use data augmentation to improve model generalization.

c. Compare model performance with and without Batch Normalization and Dropout

In [1]:
```
%pip install -q tensorflow numpy matplotlib datasets
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from datasets import load_dataset
from tensorflow.keras import layers, models

print("TensorFlow version:", tf.__version__)
```

```
2026-10-07 13:04:14.362043: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to flo
ating-point round-off errors from different computation orders. To turn them o
ff, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-10-07 13:04:14.362597: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:3
2] Could not find cuda drivers on your machine, GPU will not be used.
2026-10-07 13:04:14.365480: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:3
2] Could not find cuda drivers on your machine, GPU will not be used.
2026-10-07 13:04:14.373525: E external/local_xla/xla/stream_executor/cuda/cuda
_fft.cc:477] Unable to register cuFFT factory: Attempting to register factory
for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written t
o STDERR
E0000 00:00:1791358454.390092   27206 cuda_dnn.cc:8310] Unable to register cuD
NN factory: Attempting to register factory for plugin cuDNN when one has alrea
dy been registered
E0000 00:00:1791358454.394458   27206 cuda_blas.cc:1418] Unable to register cu
BLAS factory: Attempting to register factory for plugin cuBLAS when one has al
ready been registered
2026-10-07 13:04:14.409979: I tensorflow/core/platform/cpu_feature_guard.cc:21
0] This TensorFlow binary is optimized to use available CPU instructions in pe
rformance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
/home/admin1/anaconda3/lib/python3.9/site-packages/scipy/__init__.py:155: User
Warning: A NumPy version >=1.18.5 and <1.25.0 is required for this version of
SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}"
TensorFlow version: 2.18.0
```

In [3]:
```python
dataset = load_dataset("p2pfl/CIFAR10")

print(dataset)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
DatasetDict({
    train: Dataset({
        features: ['image', 'label'],
        num_rows: 50000
    })
    test: Dataset({
        features: ['image', 'label'],
        num_rows: 10000
    })
})
```

In [4]:
```python
train_data = dataset["train"]
test_data = dataset["test"]

x_train = np.array([np.array(image) for image in train_data["image"]])
y_train = np.array(train_data["label"])

x_test = np.array([np.array(image) for image in test_data["image"]])
y_test = np.array(test_data["label"])

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
print("Training labels shape:", y_train.shape)
print("Testing labels shape:", y_test.shape)
```

```
Training data shape: (50000, 32, 32, 3)
Testing data shape: (10000, 32, 32, 3)
Training labels shape: (50000,)
Testing labels shape: (10000,)
```

In [5]:
```
#Normalize data
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())
```

```
Minimum pixel value: 0.0
Maximum pixel value: 1.0
```

In [6]:
```
#Create ANN
ann_model = models.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Flatten(),
    layers.Dense(512, activation="relu"),
    layers.Dense(256, activation="relu"),
    layers.Dense(10, activation="softmax")
])

ann_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

ann_model.summary()
```

```
2026-10-07 13:04:31.961576: E external/local_xla/xla/stream_executor/cuda/cuda
_driver.cc:152] failed call to cuInit: INTERNAL: CUDA error: Failed call to cu
Init: UNKNOWN ERROR (303)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 3072)	0
dense (Dense)	(None, 512)	1,573,376
dense_1 (Dense)	(None, 256)	131,328
dense_2 (Dense)	(None, 10)	2,570

Total params: 1,707,274 (6.51 MB)

Trainable params: 1,707,274 (6.51 MB)

Non-trainable params: 0 (0.00 B)

In [7]:
```
#Train ANN
history_ann = ann_model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=64,
```

```
    validation_split=0.1
)
```

```
Epoch 1/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.2690 - loss: 2.0518 - v
al_accuracy: 0.3532 - val_loss: 1.8136
Epoch 2/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.3858 - loss: 1.7137 - v
al_accuracy: 0.3766 - val_loss: 1.7317
Epoch 3/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.4211 - loss: 1.6146 - v
al_accuracy: 0.4012 - val_loss: 1.6893
Epoch 4/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.4427 - loss: 1.5674 - v
al_accuracy: 0.4482 - val_loss: 1.5641
Epoch 5/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.4565 - loss: 1.5075 - v
al_accuracy: 0.4652 - val_loss: 1.5151
Epoch 6/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.4717 - loss: 1.4813 - v
al_accuracy: 0.4392 - val_loss: 1.5518
Epoch 7/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.4864 - loss: 1.4388 - v
al_accuracy: 0.4766 - val_loss: 1.4853
Epoch 8/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.4933 - loss: 1.4135 - v
al_accuracy: 0.4754 - val_loss: 1.4939
Epoch 9/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.5062 - loss: 1.3831 - v
al_accuracy: 0.4740 - val_loss: 1.4871
Epoch 10/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.5146 - loss: 1.3636 - v
al_accuracy: 0.4912 - val_loss: 1.4429
```

In [8]:
```
#Test ANN
ann_loss, ann_accuracy = ann_model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("ANN Test Loss:", ann_loss)
print("ANN Test Accuracy:", ann_accuracy)
print("ANN Test Accuracy (%):", ann_accuracy * 100)
```

```
ANN Test Loss: 1.437561273574829
ANN Test Accuracy: 0.49300000071525574
ANN Test Accuracy (%): 49.300000071525574
```

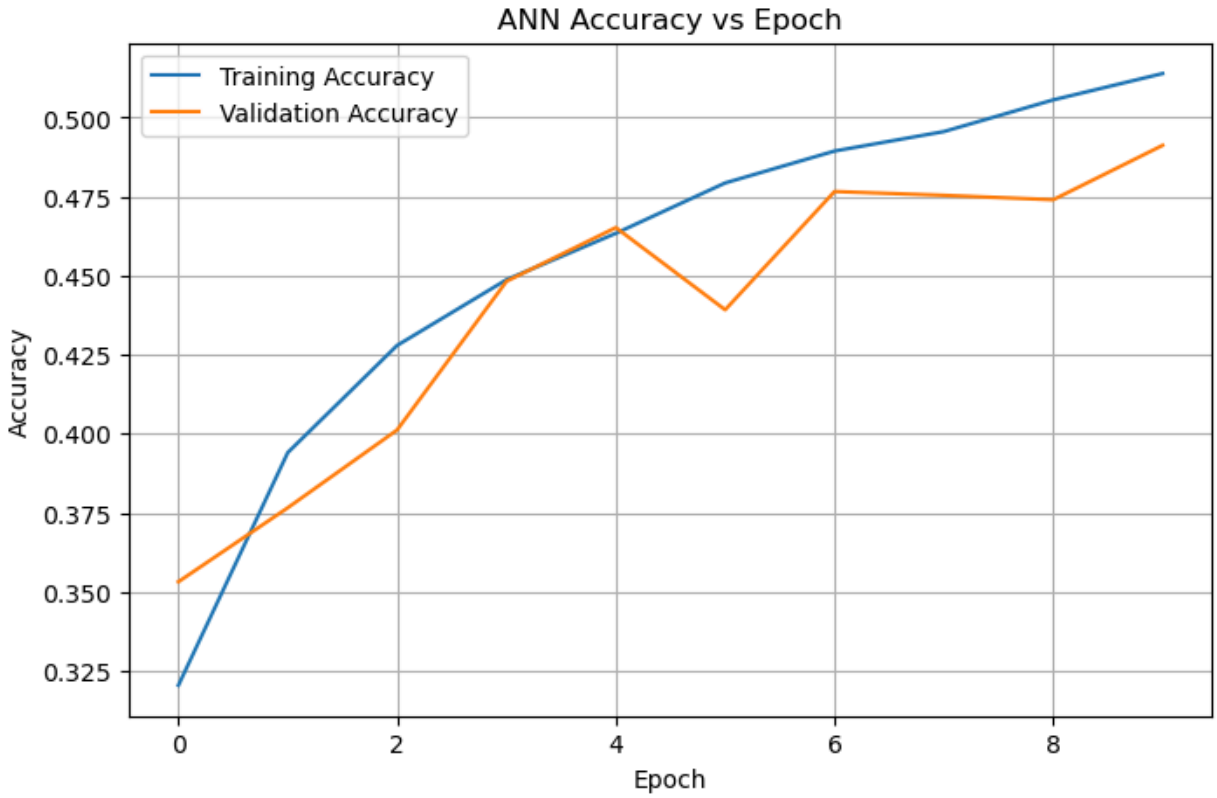
In [9]:
```
#ANN Accuracy Graph
plt.figure(figsize=(8, 5))

plt.plot(
    history_ann.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history_ann.history["val_accuracy"],
    label="Validation Accuracy"
)
```

```
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("ANN Accuracy vs Epoch")

plt.legend()
plt.grid(True)
plt.show()
```



In [10]:
```
#Create CNN
cnn_model = models.Sequential([

    # Input
    layers.Input(shape=(32, 32, 3)),

    # Convolution Layer 1
    layers.Conv2D(
        32,
        (3, 3),
        activation="relu"
    ),

    # Pooling
    layers.MaxPooling2D((2, 2)),

    # Convolution Layer 2
    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    # Pooling
    layers.MaxPooling2D((2, 2)),
```

```
    # Convolution Layer 3
    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    # Flatten
    layers.Flatten(),

    # Dense layer
    layers.Dense(
        64,
        activation="relu"
    ),

    # Output layer
    layers.Dense(
        10,
        activation="softmax"
    )
])

cnn_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

cnn_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten_1 (Flatten)	(None, 1024)	0
dense_3 (Dense)	(None, 64)	65,600
dense_4 (Dense)	(None, 10)	650

Total params: 122,570 (478.79 KB)

Trainable params: 122,570 (478.79 KB)

Non-trainable params: 0 (0.00 B)

In [11]:
```
#Train CNN
history_cnn = cnn_model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1
)
```

```
Epoch 1/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.3281 - loss: 1.8314 -
val_accuracy: 0.5172 - val_loss: 1.3416
Epoch 2/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5348 - loss: 1.2916 -
val_accuracy: 0.5742 - val_loss: 1.1856
Epoch 3/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5987 - loss: 1.1284 -
val_accuracy: 0.6374 - val_loss: 1.0412
Epoch 4/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.6399 - loss: 1.0234 -
val_accuracy: 0.6642 - val_loss: 0.9677
Epoch 5/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.6759 - loss: 0.9229 -
val_accuracy: 0.6698 - val_loss: 0.9580
Epoch 6/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.6978 - loss: 0.8642 -
val_accuracy: 0.6782 - val_loss: 0.9310
Epoch 7/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.7159 - loss: 0.8107 -
val_accuracy: 0.6902 - val_loss: 0.8889
Epoch 8/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.7337 - loss: 0.7627 -
val_accuracy: 0.7080 - val_loss: 0.8497
Epoch 9/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.7508 - loss: 0.7137 -
val_accuracy: 0.7218 - val_loss: 0.8115
Epoch 10/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.7679 - loss: 0.6621 -
val_accuracy: 0.7288 - val_loss: 0.8034
```

In [12]:
```
#Test CNN
cnn_loss, cnn_accuracy = cnn_model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("CNN Test Loss:", cnn_loss)
print("CNN Test Accuracy:", cnn_accuracy)
print("CNN Test Accuracy (%):", cnn_accuracy * 100)
```

```
CNN Test Loss: 0.8246912360191345
CNN Test Accuracy: 0.7200000286102295
CNN Test Accuracy (%): 72.00000286102295
```

In [13]:
```
plt.figure(figsize=(8, 5))

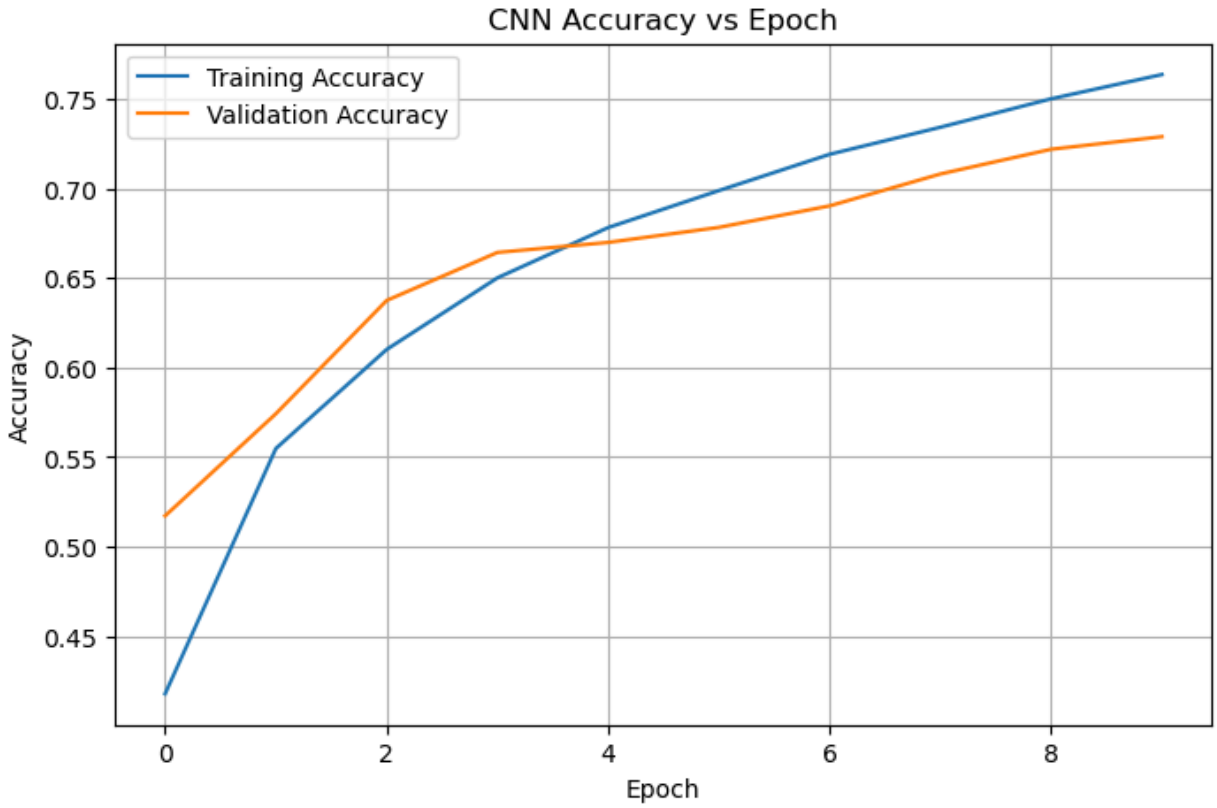
plt.plot(
    history_cnn.history["accuracy"],
    label="Training Accuracy"
```

```
)

plt.plot(
    history_cnn.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Accuracy vs Epoch")

plt.legend()
plt.grid(True)
plt.show()
```



In [17]:

```
#CNN Loss Graph
plt.figure(figsize=(8, 5))

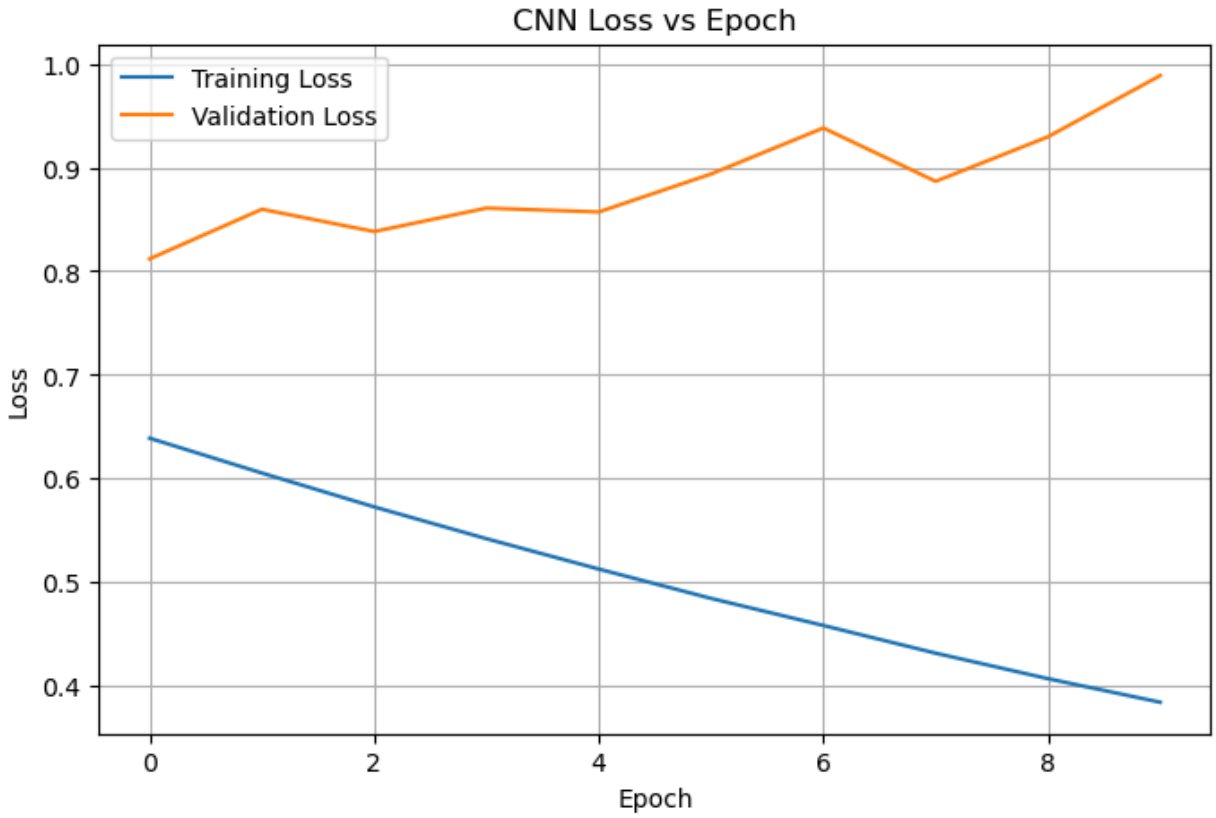
plt.plot(
    history_cnn.history["loss"],
    label="Training Loss"
)

plt.plot(
    history_cnn.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("CNN Loss vs Epoch")

plt.legend()
```

```
plt.grid(True)
plt.show()
```



In [25]:
```
#Create improved CNN
cnn_bn_dropout_model = models.Sequential([

    # Convolution Layer 1
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(
        32,
        (3, 3),
        padding="same",
        activation="relu"
    ),

    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    # Convolution Layer 2
    layers.Conv2D(
        64,
        (3, 3),
        padding="same",
        activation="relu"
    ),

    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    # Convolution Layer 3
    layers.Conv2D(
        128,
        (3, 3),
```

```
        padding="same",
        activation="relu"
    ),

    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    # Flatten
    layers.Flatten(),

    # Dense layer
    layers.Dense(
        128,
        activation="relu"
    ),

    # Batch Normalization
    layers.BatchNormalization(),

    # Dropout
    layers.Dropout(0.5),

    # Output layer
    layers.Dense(
        10,
        activation="softmax"
    )
])

cnn_bn_dropout_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

cnn_bn_dropout_model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 32, 32, 32)	896
batch_normalization_8 (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d_8 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_10 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_9 (BatchNormalization)	(None, 16, 16, 64)	256
max_pooling2d_9 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_11 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_10 (BatchNormalization)	(None, 8, 8, 128)	512
max_pooling2d_10 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten_4 (Flatten)	(None, 2048)	0
dense_9 (Dense)	(None, 128)	262,272
batch_normalization_11 (BatchNormalization)	(None, 128)	512
dropout_2 (Dropout)	(None, 128)	0
dense_10 (Dense)	(None, 10)	1,290

Total params: 358,218 (1.37 MB)

Trainable params: 357,514 (1.36 MB)

Non-trainable params: 704 (2.75 KB)

In [19]:
```
#Train the improved CNN
history_cnn_bn_dropout = cnn_bn_dropout_model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1
)
```

```
Epoch 1/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 21s 26ms/step - accuracy: 0.4187 - loss: 1.8140 -
val_accuracy: 0.5574 - val_loss: 1.2402
Epoch 2/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 18s 25ms/step - accuracy: 0.6385 - loss: 1.0249 -
val_accuracy: 0.6684 - val_loss: 0.9472
Epoch 3/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 19s 27ms/step - accuracy: 0.7090 - loss: 0.8417 -
val_accuracy: 0.6656 - val_loss: 0.9615
Epoch 4/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 19s 27ms/step - accuracy: 0.7486 - loss: 0.7242 -
val_accuracy: 0.7208 - val_loss: 0.8041
Epoch 5/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 19s 26ms/step - accuracy: 0.7777 - loss: 0.6465 -
val_accuracy: 0.7006 - val_loss: 0.8821
Epoch 6/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 18s 26ms/step - accuracy: 0.8055 - loss: 0.5586 -
val_accuracy: 0.5616 - val_loss: 1.4357
Epoch 7/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 18s 26ms/step - accuracy: 0.8222 - loss: 0.5229 -
val_accuracy: 0.7596 - val_loss: 0.7172
Epoch 8/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 18s 25ms/step - accuracy: 0.8467 - loss: 0.4397 -
val_accuracy: 0.7584 - val_loss: 0.7486
Epoch 9/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 17s 24ms/step - accuracy: 0.8647 - loss: 0.3934 -
val_accuracy: 0.7468 - val_loss: 0.7894
Epoch 10/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 18s 25ms/step - accuracy: 0.8737 - loss: 0.3663 -
val_accuracy: 0.7254 - val_loss: 0.9745
```

In [20]:

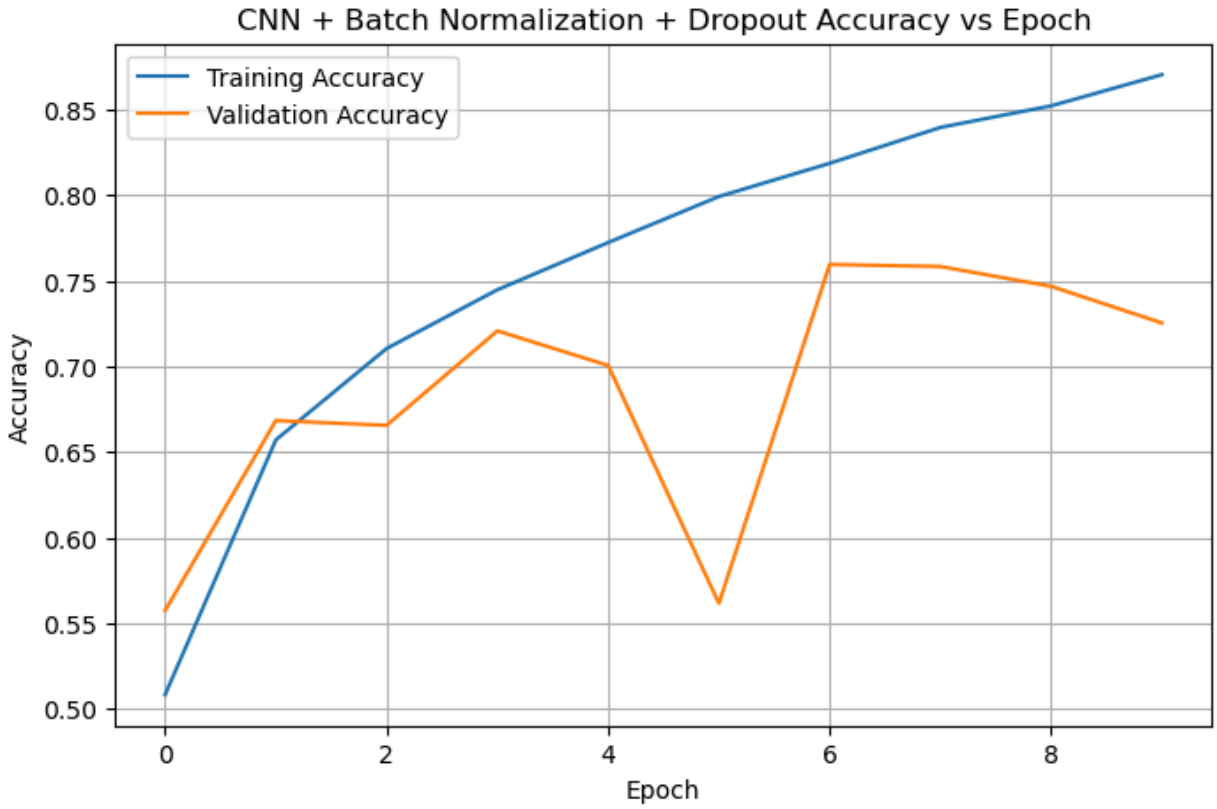
```python
#Accuracy Graph
plt.figure(figsize=(8, 5))

plt.plot(
    history_cnn_bn_dropout.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history_cnn_bn_dropout.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title(
    "CNN + Batch Normalization + Dropout Accuracy vs Epoch"
)

plt.legend()
plt.grid(True)
plt.show()
```



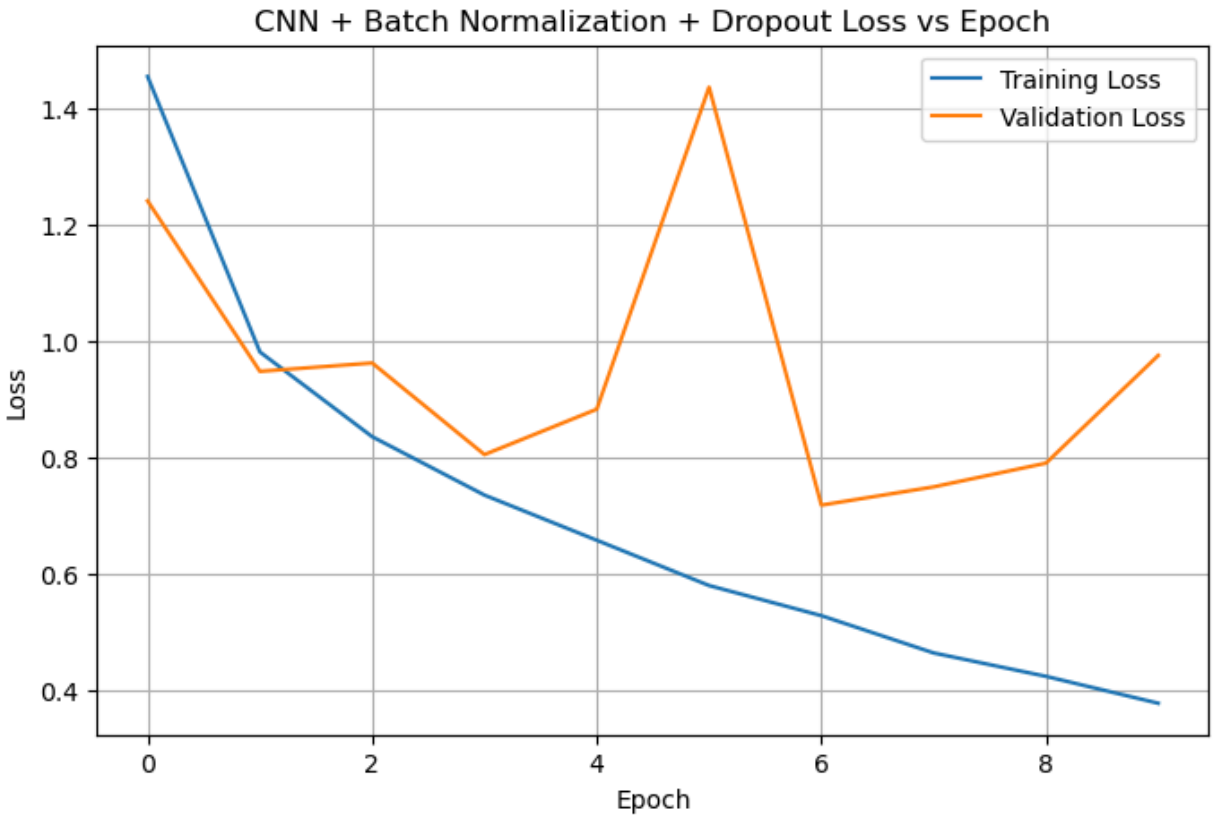
In [21]:
```
#Loss Graph
plt.figure(figsize=(8, 5))

plt.plot(
    history_cnn_bn_dropout.history["loss"],
    label="Training Loss"
)

plt.plot(
    history_cnn_bn_dropout.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title(
    "CNN + Batch Normalization + Dropout Loss vs Epoch"
)

plt.legend()
plt.grid(True)
plt.show()
```



In [22]:
```
#Test improved CNN
cnn_bn_dropout_loss, cnn_bn_dropout_accuracy = (
    cnn_bn_dropout_model.evaluate(
        x_test,
        y_test,
        verbose=0
    )
)

print(
    "CNN + Batch Normalization + Dropout Test Loss:",
    cnn_bn_dropout_loss
)

print(
    "CNN + Batch Normalization + Dropout Accuracy:",
    cnn_bn_dropout_accuracy
)

print(
    "CNN + Batch Normalization + Dropout Accuracy (%):",
    cnn_bn_dropout_accuracy * 100
)
```

```
CNN + Batch Normalization + Dropout Test Loss: 0.9974438548088074
CNN + Batch Normalization + Dropout Accuracy: 0.7166000008583069
CNN + Batch Normalization + Dropout Accuracy (%): 71.66000008583069
```

In [23]:
```
#Comparision
print("=" * 60)
print("MODEL PERFORMANCE COMPARISON")
print("=" * 60)
```

```
print(f"ANN Accuracy: {ann_accuracy * 100:.2f}%")

print(f"CNN Accuracy: {cnn_accuracy * 100:.2f}%")

print(
    f"CNN + BN + Dropout Accuracy: "
    f"{cnn_bn_dropout_accuracy * 100:.2f}%"
)
```

```
============================================================
MODEL PERFORMANCE COMPARISON
============================================================
ANN Accuracy: 49.30%
CNN Accuracy: 72.00%
CNN + BN + Dropout Accuracy: 71.66%
```

In [24]:

```
#Comparision Graph
models_name = [
    "ANN",
    "CNN",
    "CNN + BN + Dropout"
]

accuracies = [
    ann_accuracy * 100,
    cnn_accuracy * 100,
    cnn_bn_dropout_accuracy * 100
]

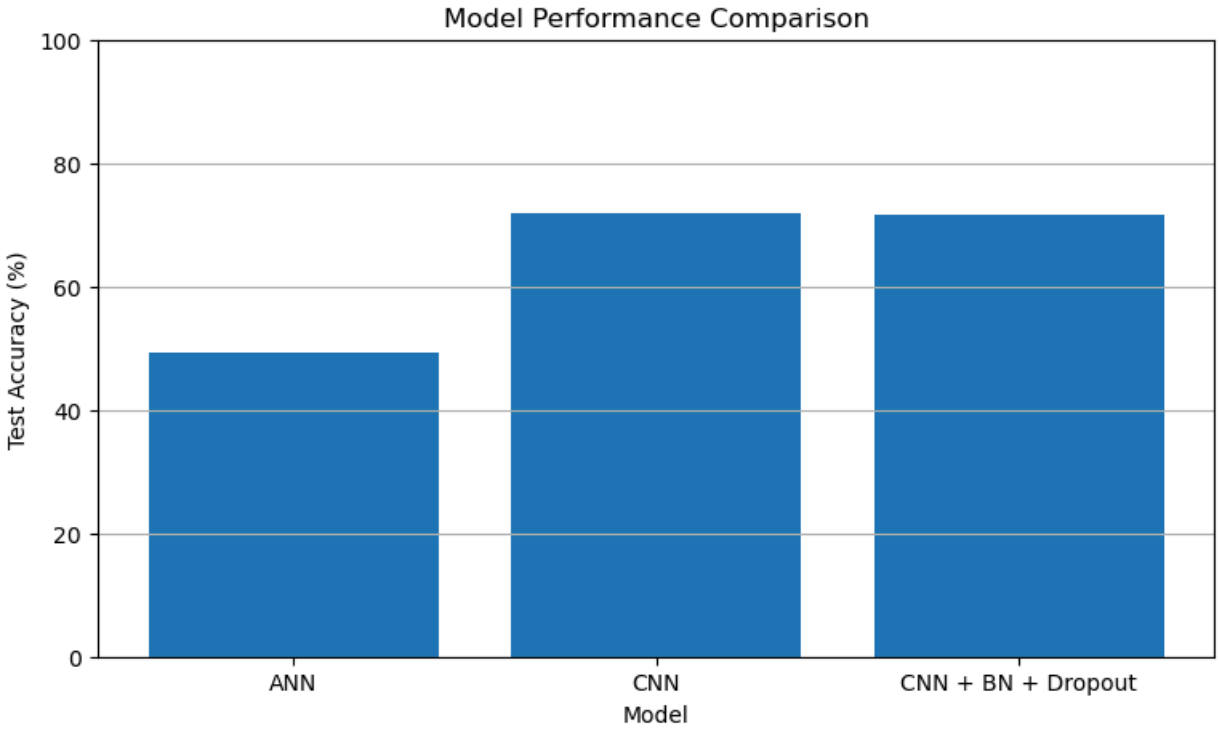
plt.figure(figsize=(9, 5))

plt.bar(models_name, accuracies)

plt.xlabel("Model")
plt.ylabel("Test Accuracy (%)")
plt.title("Model Performance Comparison")

plt.ylim(0, 100)
plt.grid(axis="y")

plt.show()
```


Model Performance Comparison
Test Accuracy (%)
100
80
60
40
20
0
ANN
CNN
CNN + BN + Dropout
Model

In [ ]: